Topic 2: Software Configuration Management Disciplines

Introduction

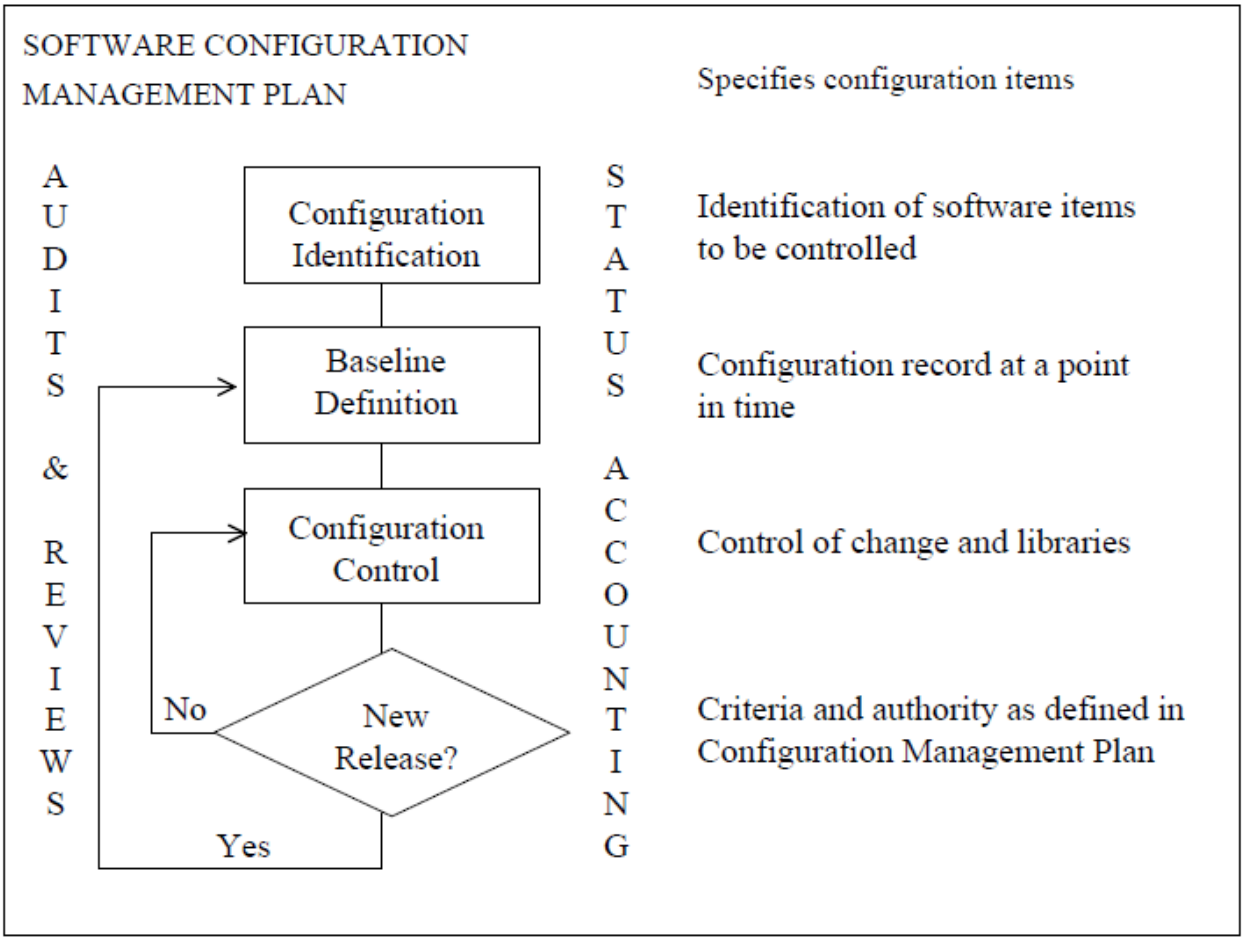
SCM disciplines are used to simultaneously coordinate configurations of many different representations of the software product (e.g., source code, relocatable code, executable code, and documentation).

In practice, the ways in which SCM disciplines are performed will be different for the various kinds of software being developed (e.g., commercial, embedded, original equipment manufacturer).

The degree of formal documentation required within and across different lifecycle management phases (e.g., research, product development, operations, and maintenance) will also vary.

The effectiveness of the SCM system increases in proportion to the degree that its disciplines are an explicit part of the normal day-to-day activities of those involved in the software development and maintenance efforts.

Figure below presents a diagram showing how the SCM disciplines interact at the highest level.



The Software Configuration Management Plan (SCM Plan) provides information on the requirements and procedures necessary for the configuration management activities of the software engineering project.

It specifies who will be responsible for accomplishing the planned activities, what activities will be performed, when the activities will be performed in coordination with the project schedule, and what tools and human resources will be required to perform the activities.

The first step in the SCM system is to identify the Configuration Items to be controlled and, as the project progresses, to identify the structure of the intermediate or complete software products. Elements to be controlled will normally include the software code and associated documentation. Documents could include requirements specifications, designs, user guides, test suites, test results, and user lists.

Baselines are defined and created in accordance with the SCM Plan and the phase of the project's lifecycle.

A **baseline** is a specification or product that has been formally reviewed and agreed upon, that serves as the basis for further development, and can only be changed through formal change control procedures.

A **baseline** is like a snapshot of the aggregate of software components as they exist at a given point in time.

During each phase of the software lifecycle, configuration items that are completed and accepted by the approval authority become the baseline for that phase.

Any addition, alteration, or deletion to the approved baseline is deemed a change, and is subject to change control.

Baselines, plus the approved changes from those baselines, constitute the current configuration identification.

Each of the baselines established during a software lifecycle controls subsequent software development. The following are typical configuration baselines that are established during the life of a software project.

- Functional Baseline - Established by the acceptance or approval of the software or system specification. This baseline typically corresponds to the completion of the software requirement review.

- Allocated Baseline – Established by approval of the software requirements specification. This baseline typically corresponds to the completion of the software specification review.

- Developmental Baseline – Established by the approval of the technical documentation that defines the functional and detailed design (including documentation of interfaces and databases for the computer software). Normally, this baseline corresponds to the time frame spanning the preliminary design review and the critical design review.

- Product Baseline – Established by approval of the product specification following completion of the last formal functional configuration audit (FCA).

Once configuration items have been identified and baselined, Configuration Control provides a system to initiate, review, approve, and implement proposed changes. Each engineering change or problem report that is initiated against a formally identified configuration item is evaluated to determine its necessity and impact. Approved changes are implemented, testing is performed to verify that the change was correctly implemented and that no unexpected side effects have occurred as a result of the change, and the affected documentation is updated and reviewed.

The discipline of Status Accounting collects information about all configuration management activities. The status account should be capable of providing data on the status and history of any configuration item. The information maintained in Status Accounting should enable the rebuild of any previous baseline.
Audits and reviews are conducted to verify that the software product matches the configuration item descriptions in the specifications and documents, and that the package being reviewed is complete. Any anomalies found during audits should be corrected and the root cause of the problem identified and corrected to ensure that the problem does not resurface.
The audit must also establish that the correct version and revision of each part are included in the product baseline and that they correspond to information contained in the baseline's configuration status report.

Configuration Identification

The first step in the SCM system is the identification of the items to be managed and the specification of the management authority responsible for each item.
This is a critical step in the system since the identification scheme is employed throughout the lifecycle of the software product. The levels of authority assigned responsibility for each configuration item will vary depending on the complexity of the software product, the size of the development team, and the management style of the responsible organization.
Configuration identification consists of selecting the configuration items and recording their functional and physical characteristics as set forth in technical documentation such as specifications, drawings, and associated lists. The following are examples of items managed in the SCM system.

- Management plans
- Specifications (requirements, design)
- User documentation
- Test plans, test design, case, and procedure specifications
- Test data and test generation procedures
- Support software used in development, even though not part of the delivered software product
- Data dictionaries and various cross-references
- Source code (on machine-readable media)
- Executable code (the run-time system)
- Libraries
- Databases (data that are processed and data that are part of a program)
- Maintenance documentation (e.g., listings, detail design descriptions)

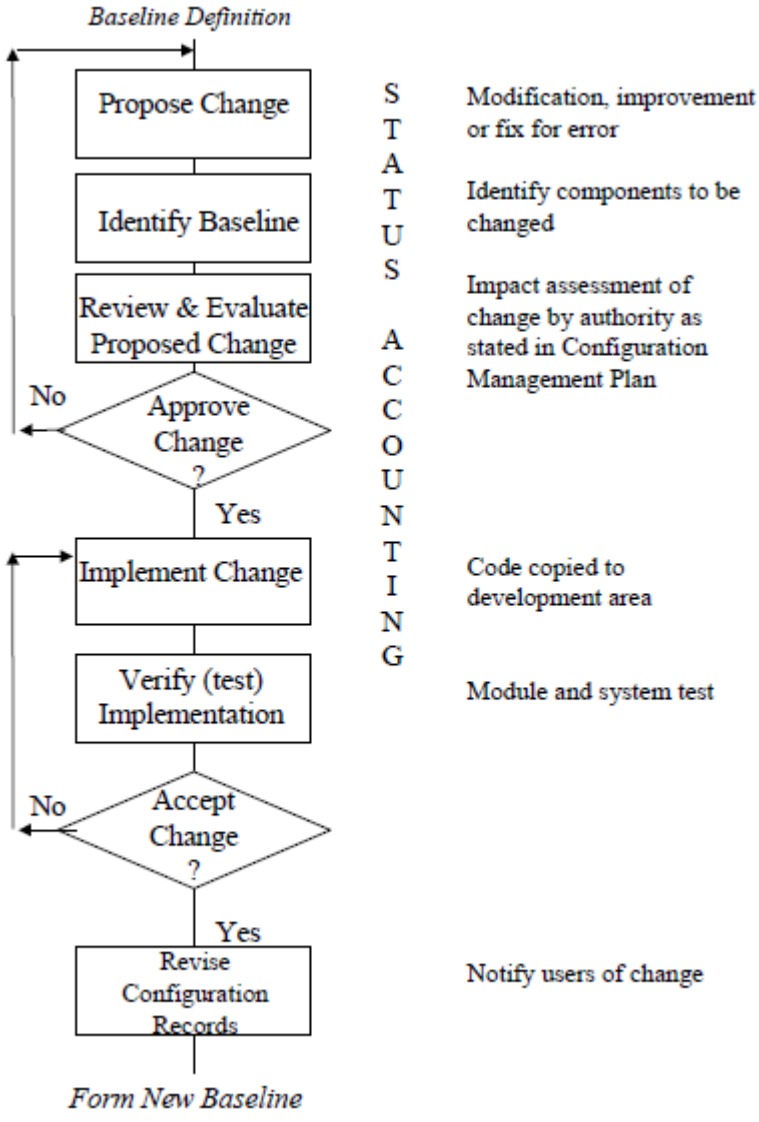
Effective management of the development of a software product requires careful definition of the configuration items.
Changes to these items also need to be defined since these changes, along with the project baselines, specify the evolution of the software product. SCM includes all of the entities of the software product as well as their various representations in documentation. The entities are not all subject to the same SCM disciplines at the same time.

Configuration Control (Change Control)

The primary function of configuration control is to provide the administrative mechanism for initiating, preparing, evaluating, and approving or disapproving all change proposals throughout the software lifecycle.

Configuration control ensures that changes to configuration items are controlled and that consistency between component parts of a system is maintained. It reduces the possibility of making changes that may later adversely affect functionality.

Figure below provides a diagram of the configuration control process.



The configuration control process involves three elements:

- Procedures for controlling changes to a software product.
- Levels of authority (e.g., Configuration Control Board) for formally evaluating and approving or disapproving proposed changes to the software.
- Documentation, such as administrative forms and supporting technical and administrative material, for formally initiating and defining a proposed change to a software system.

Many automated tools support the configuration control process. The major ones aid in controlling software change once the coding stage has been reached. Automation of other functions, such as library access control, software and document version maintenance, change recording, and document reconstruction, greatly enhance both the control and maintenance processes.

Procedures for Controlling Changes

Changes occur at all phases in the software lifecycle. Design or implementation changes may be necessary if requirements change, or when deficiencies are identified in the design or implementation approach to a stable requirement.

Testing may uncover defects that require changes in the code or the design and requirements. Changes must be made to the right version of the code, testing must verify performance of the change and the integrity of the remaining software, and all associated documentation must be updated to be consistent with the final code.

A mechanism is needed to process change requests (e.g., discrepancies, failure reports, requests for new features) from a variety of sources throughout the development process and during operation and support of the software product.

This mechanism should extend to a definition of a process to track, evaluate, and implement change requests into a new version and new release of the software. Generally, no single procedure can meet the needs of all levels of change management and approval levels.

The minimum activities needed for processing changes include:

- Defining the information needed for approving the requested change.
- Identifying the review process and routing of information.
- Describing the control of the libraries used to process the change.
- Developing a procedure for implementing each change in the code, the documentation, and the released software product.

Software Libraries (Repositories)

Software libraries are a controlled collection of software and related documentation designed to aid in software development, use, and maintenance.

Software libraries provide the means for identifying and labeling baselined entities, and for capturing and tracking the status of changes to those entities.

Libraries have been historically composed of documentation on hard copy and software on machine-readable media, but the trend is moving towards all information being created and maintained on machine-readable media.

This trend, which encourages the increased use of automated tools, leads to higher productivity. The trend also means that the libraries are a part of the software engineering working environment. The SCM functions associated with the libraries have to become part of the software engineering environment, making the process of configuration management more transparent to the software developers and maintainers.

Levels of Authority

The levels of authority required for approving changes to configuration items under SCM control can vary.

The level of control needed to authorize changes depends on the level of the item in relation to the product as a whole. The system or contractual requirements may dictate the level of authority needed. For example, internally controlled software tools typically require less change controls than critical software.

Levels of authority can vary throughout the software lifecycle. For example, changes to code in the development cycle usually require a lower level of control to be authorized than changes to the same code after it has been released for general use.

The level of authority can also depend on how broadly the change impacts the system. A change affecting specifications during the requirements analysis phase has less impact than a change affecting software in operational use.

Configuration Control Boards

Configuration Control Boards (CCBs) enable the implementation of change controls at optimum levels of authority and scope. CCBs can exist in a hierarchical fashion (for example, at the program, system design, and program product level), or one board may have authority over all levels of the change process.

In most organizations, the CCB is composed of senior level managers. They include representatives from the major software, hardware, test, engineering, and support organizations as defined in the SCM Plan. The purpose of the CCB is to control major issues such as schedule, function, and the configuration of the system as a whole. The CCB functions may be included in formal computer-aided software engineering (CASE) tools.

Some of the functions of the CCB include:

- Directing the entire configuration management effort.
- Resolving all problems and situations that arise during the effort.
- Using the SCM system as its primary decision-making resource.
- Taking into account organizational management considerations for decision making.
- Orchestrating all activities from the beginning to the approval of the baselines for SCM establishment.
- Determining which products should be baselined or managed, the method to be used, and the order in which they should be done.
- Resolving all issues such as the most suitable product name to be used from among numerous documented aliases. Multiple aliases often occur when an SCM system was not originally in place.

Software Configuration Control Boards

The more technical issues that do not relate to issues of performance, cost, and schedule are often assigned to a Software Configuration Control Board (SCCB).

The SCCB discusses issues related to specific schedules for partial functions, interim delivery dates, common data structures, design changes, and the like.

This is the place for decision-making concerning the items that must be coordinated across configuration items, but they do not require the attention of high-level management.

The SCCB members should be technically well versed in the details of their area, while the CCB members are more concerned with broad management issues facing the project as a whole and with customer issues.
Depending on the size and nature of the software project, the SCCB may consist of a single librarian, multiple librarians, or include many SCCBs, each with a different functional responsibility for ensuring that changes are implemented and tested according to standard procedures and that hardware/software interfaces and interfaces between software modules are not violated.
The SCCB also focuses on overall project management responsibility for ensuring that design or requirements specifications are not violated and that software changes are implemented according to cost and schedule constraints. The size and structure of SCCBs normally change over the lifecycle of the software.

Documentation

The Software Change Request (SCR) is one of the major tools used for identifying and coordinating changes to software and documentation.
The SCR is used to capture important information about the change and to track the status of a change from its proposal to its eventual disposition.
A typical SCR form contains a narrative description of the change or problem, information to identify the source of the report, and some basic information to aid in evaluating the report.
An SCR is submitted only against baselined software or documentation. An SCR may be submitted by anyone associated with the project, but usually is submitted by a member of the software development team.
The Software Change Authorization (SCA) form is used to control changes to all software and documents under SCM control and software and documents that have been released to SCM.
The SCA form is used by software developers to submit changes to software and documents to SCM in order to update the master library.
Several reports may be needed to support the configuration status accounting needs. Typical reports include a list of all SCRs by status (e.g., open, closed); a monthly summary of the SCRs and SCAs; all SCRs submitted per unit or software component within a selected range of submittal dates; the status of all documentation under SCM control; and a version description document. [3]

Configuration Status Accounting

The formal process of tracking software entities through the steps in their evolution is referred to as Status Accounting or Configuration Status Accounting (CSA). Status Accounting provides the project manager with the data necessary to gauge and report project progress.
Status Accounting provides a mechanism for maintaining a record of how the software has evolved and where the software is at any time relative to the information contained in baseline documents. Status Accounting answers the following questions:
(1) What happened? (2) Who did it? (3) When did it happen? (4) What else will be affected?

Status Accounting is a function that increases in complexity as the software lifecycle progresses because of the multiple software representations that emerge with later baselines. This complexity generally results in larger amounts of data to be recorded and reported.

The purpose of Status Accounting is to have the ability, at any point in the software lifecycle, to provide the current content of a given software configuration item or computer software component. If the entity is a software design description, a status accounting mechanism should allow developers to easily pull together any text files, graphic files, and data files that may comprise the document. Likewise, change requests to software products should be easy to track and the status of these requests should be readily reproducible in an organized manner.

Each time a software configuration item is assigned a new or updated identification, a status report entry is made. Each time a change is approved, a status report entry is made. Each time a configuration audit is conducted, the results are reported as part of a status reporting task.

Output from a status accounting report may be placed in an online database so that software developers or maintainers can access change information by key-word category.

Status accounting reports are generated on a regular basis and are intended to keep management and practitioners appraised of important changes.

Status accounting reports should be maintained in electronic files at logical transition points in the software lifecycle. For example, when a baseline configuration is advanced from the design phase to an implementation (coding) phase, a record of this should be filed in the configuration records. Similarly, when a newly developed or modified module has been approved for system testing and integration, this transition in status should be recorded.

Audits and Reviews

Audits and reviews are performed to verify that the software product matches the capabilities defined in the specifications or other contractual documentation and that the performance of the product fulfills the requirements of the user or customer.

Audits

Software configuration audits provide the mechanism for determining the degree to which the current state of the software mirrors the software depicted in baseline and requirements documentation. Software auditing increases software visibility and establishes the traceability of changes throughout the lifecycle.

It reveals whether the project requirements are being satisfied and whether the intent of the preceding baseline has been fulfilled.

The audits enable project management to evaluate the integrity of the software product being developed, resolve issues that may have been raised by the audit, and correct defects in the development process.

There are two types of audits that should be performed prior to release of a product baseline or a revision of an existing baseline:

Physical Configuration Audit (PCA)

A PCA is performed to determine whether all items identified as being part of the configuration are present in the product baseline.

The audit must establish that the correct version and revision of each part are included in the product baseline and that they correspond to information contained in the baseline's configuration status report.

Functional Configuration Audit (FCA)

An FCA is conducted to ensure that each item of the software product has been tested or inspected to determine that it satisfies the functions defined in the specifications. An important aspect of these two audits is the assurance that all documentation (change requests, test data, and reports, etc.) relevant to the release are current and complete.

In large software projects, audits may be necessary throughout the evolution of a product to ensure conformance with planned configuration management actions and to prevent significant problems from being encountered just prior to release.

Technical Reviews

Formal technical reviews are performed throughout the project, although SCM disciplines generally do not initiate or direct reviews. Quite often the mechanisms used by SCM to process changes are used to organize and process items in a review conducted by other functions such as Software Quality Assurance. SCM supports reviews and provides the mechanism for acting on changes resulting from the reviews.

Release Processing

Often the recipient will need installation instructions and other data concerning the release.

The installation instructions should define the environment in which the software product will run—both hardware and software.

The release package typically includes documentation (often referred to as the version description document). The more critical or the larger the software product, the more complete the documentation needs to be. SCM verifies that the release package is complete and ready to be delivered.

Once a release is completed, a configuration baseline should be captured that associates components and their versions with the release identity of the whole product.